

Entretien Technique

Backend Python

[Django](#) · [FastAPI](#) · [DRF](#) · [PostgreSQL](#) · [Tests](#) · [Architecture](#)

Plus de 300 questions–reponses classees
Junior → Mid → Senior

JUNIOR

MID

SENIOR

Support de revision a imprimer

Table des matières

1	Comment utiliser ce document	2
2	Python Core	3
2.1	Types de donnees	3
2.2	Variables, mutabilite, copies	3
2.3	Fonctions	4
2.4	Programmation orientee objet	5
2.5	Methodes speciales (dunder)	6
2.6	Decorateurs	7
2.7	Generateurs et iterateurs	8
2.8	Gestion memoire	8
2.9	Exceptions	9
2.10	Async Python	10
3	Django	11
3.1	Architecture	11
3.2	Routing, vues, middleware	11
3.3	ORM	12
3.4	Relations, migrations, admin, signals, forms	13
4	Django REST Framework (DRF)	14
5	FastAPI	15
5.1	Pydantic	15
5.2	Dependances, requetes, reponses	16
6	REST API	18
7	SQL	19
8	PostgreSQL	21
9	Securite	22
10	Tests	24
11	Docker	26
12	Git	27
13	Architecture logicielle	28
14	Performance	29
15	Deploiement	30
16	Questions sur tes projets	31

1. Comment utiliser ce document

Ce manuel couvre les questions qui reviennent le plus souvent en entretien pour un poste **Backend Software Engineer Python** (Django et/ou FastAPI), sur la base d’une synthèse de sources 2025–2026 (TechPrep, Learnix, HackerX, Medium, Reddit, Baeldung).

Repartition observee d’un entretien type

Python core $\approx 35\%$		Framework (Django/FastAPI) $\approx 25\%$		Base de donnees / SQL $\approx 15\%$		Tests, securite, archi, deploiement $\approx 10\%$		Questions sur tes projets $\approx 30\text{--}40\%$
----------------------------	--	---	--	--------------------------------------	--	--	--	---

Niveaux. Chaque question porte un badge : **JUNIOR** (bases attendues de tous), **MID** (comprehension pratique / production), **SENIOR** (architecture, performance, decisions de conception).

Conseil de revision. Pour un profil deja experimente (FastAPI, RAG, Kafka, Spark, PostgreSQL, microservices), concentre-toi en priorite sur les sections *Async Python*, *FastAPI en production*, *Django ORM & performance*, *PostgreSQL*, *Securite* et *Questions projets* – les bases Python doivent rester des reflexes, pas une revision.

2. Python Core

Types de donnees

JUNIOR Q1. Quelle est la difference entre une `list` et un `tuple` ?

Reponse. Les deux sont des sequences ordonnees, mais la `list` est *mutable* (on peut ajouter/retirer/modifier des elements) alors que le `tuple` est *immutable* (sa taille et son contenu sont fixes apres creation). Consquence pratique : un tuple est hashable (si ses elements le sont) et peut donc servir de cle de dictionnaire ou d'element d'un `set`, contrairement a une liste. Les tuples sont aussi legerement plus rapides a creer et consomment moins de memoire.

JUNIOR Q2. Pourquoi un tuple est-il immutable ? Quel interet concret ?

Reponse. L'immuabilite est une decision de conception : elle garantit qu'une structure ne changera pas apres sa creation, ce qui la rend sure a partager entre plusieurs parties d'un programme (pas d'effet de bord cache) et hashable (donc utilisable comme cle de `dict` ou element de `set`). C'est aussi la base du *unpacking* multiple valeurs de retour (`return x, y`).

JUNIOR Q3. Quand utiliser un `set` plutot qu'une `list` ?

Reponse. Des que l'on a besoin (1) de tester l'appartenance frequemment – `in` est en $O(1)$ moyen sur un `set` contre $O(n)$ sur une liste – ou (2) d'eliminer des doublons, ou (3) de faire des operations ensemblistes (union `|`, intersection `&`, difference `-`). Contrepartie : un `set` n'est pas ordonne et ses elements doivent etre hashables.

MID Q4. Que signifie « hashable » et pourquoi les cle de dict doivent-elles l'etre ?

Reponse. Un objet est hashable s'il possede une methode `__hash__` qui renvoie une valeur entiere stable pendant toute sa duree de vie, utilisee par le dictionnaire pour placer/retrouver la cle en $O(1)$ moyen dans sa table de hachage. Si l'objet est mutable (comme une liste), son hash pourrait changer apres modification, ce qui casserait la structure interne du dict – c'est pourquoi les listes ne sont pas hashables mais les tuples le sont (si leur contenu l'est aussi).

MID Q5. Difference entre `set` et `frozenset` ?

Reponse. `frozenset` est la version immutable de `set` : une fois cree, on ne peut plus y ajouter/retirer d'elements. Il est donc hashable et peut servir de cle de dictionnaire ou d'element d'un autre set, ce qu'un `set` classique ne permet pas.

Variables, mutabilite, copies

JUNIOR Q6. Pourquoi une liste passee en argument de fonction peut-elle etre modifiee par la fonction ?

Reponse. En Python, les arguments sont passes « par reference d'objet » (*pass by object reference*) : la variable locale de la fonction pointe vers le *me*me objet que l'appelant. Si cet objet est mutable (liste, dict, set) et que la fonction le modifie *en place* (`.append()`, `[i]=...`), le changement est visible a l'exterieur. En revanche, reassigner le parametre (`param = []`) ne fait que rebrancher la variable locale sur un nouvel objet, sans effet sur l'appelant.

```
def ajoute(l):  
    l.append(4)           # modifie l'objet partage -> visible dehors
```

```
def remplace(l):
    l = [9, 9, 9]      # rebranche la variable locale -> invisible dehors
```

MID Q7. Difference entre *shallow copy* et *deep copy*?

Reponse. Une copie superficielle (`copy.copy()`, ou `list(l), l[:]`) cree un nouvel objet conteneur mais dont les elements internes restent des *references* vers les memes objets que l'original – si un element est lui meme mutable (liste imbriquee), le modifier affecte les deux copies. Une copie profonde (`copy.deepcopy()`) reconstruit recursivement tous les objets imbriques, donc les deux structures deviennent totalement independantes.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)      # shallow
c = copy.deepcopy(a)  # deep
b[0].append(99)
print(a) # [[1, 2, 99], [3, 4]] -> a est impacte (meme sous-liste)
print(c) # [[1, 2], [3, 4]] -> c est independant
```

JUNIOR Q8. Quelle est la difference entre `==` et `is`?

Reponse. `==` compare l'*egalite de valeur* (appelle `__eq__`); `is` compare l'*identite* – est-ce le meme objet en memoire (memes `id()`)? Deux listes contenant les memes elements sont `==` mais pas `is`. On utilise `is` typiquement pour comparer a `None`, `True/False` (`if x is None`), car ce sont des singletons.

MID Q9. Pourquoi `a is b` peut etre `True` pour deux petits entiers identiques mais `False` pour de grands entiers?

Reponse. CPython met en cache (*interning*) les petits entiers entre `-5` et `256` ainsi que certaines chaines courtes : deux references vers la valeur `5` pointent donc vers le meme objet. Au-dela de cette plage, chaque litteral entier cree potentiellement un nouvel objet, donc `is` peut renvoyer `False` meme si les valeurs sont egales. C'est un detail d'implementation qu'il ne faut jamais utiliser comme mecanisme de comparaison volontaire – toujours utiliser `==` pour comparer des valeurs.

Fonctions

JUNIOR Q10. A quoi servent `*args` et `**kwargs`?

Reponse. `*args` collecte un nombre variable d'arguments positionnels dans un `tuple`; `**kwargs` collecte les arguments nommes restants dans un `dict`. Ils permettent d'ecrire des fonctions a signature flexible (wrappers, decorateurs, fonctions generiques) sans connaitre a l'avance le nombre de parametres.

```
def f(*args, **kwargs):
    print(args)      # (1, 2, 3)
    print(kwargs)    # {'x': 10}

f(1, 2, 3, x=10)
```

JUNIOR Q11. Difference entre argument positionnel et *keyword argument*?

Reponse. Un argument positionnel est associe au parametre selon sa position dans l'appel; un *keyword argument* est passe explicitement par **nom=valeur**, ce qui rend l'appel plus lisible et permet de sauter des parametres ayant une valeur par default, quel que soit leur ordre de declaration (tant qu'ils viennent apres les arguments positionnels).

MID Q12. Quel est le piege classique des arguments par default mutables ?

Reponse. Les valeurs par default sont evaluees **une seule fois**, a la definition de la fonction, et partagees entre tous les appels. Utiliser une liste ou un dict mutable comme valeur par default cree donc un etat partage et persistant entre appels – un bug tres classique.

```
def ajoute(item, liste=[]):    # BUG : meme liste reutilisee a chaque appel
    liste.append(item)
    return liste

def ajoute_ok(item, liste=None):
    if liste is None:
        liste = []
    liste.append(item)
    return liste
```

Piege frequent

`ajoute(1)` puis `ajoute(2)` renvoie `[1, 2]` et non `[2]` : c'est la meme liste par default qui s'accumule silencieusement.

Programmation orientee objet

JUNIOR Q13. Rappelle les quatre piliers de l'OOP.

Reponse. **Encapsulation** : regrouper donnees et methodes, limiter l'accès direct aux details internes (convention `_protected` / `__private`). **Heritage** : une classe fille reutilise/etend le comportement d'une classe mere. **Polymorphisme** : des objets de classes differentes repondent au meme appel de methode chacun a sa maniere (duck typing en Python). **Abstraction** : exposer une interface simple en masquant la complexite d'implementation (classes abstraites, ABC).

JUNIOR Q14. Quelle difference entre `@staticmethod` et `@classmethod` ?

Reponse. Une `@classmethod` recoit automatiquement la classe en premier argument (`cls`) et peut donc acceder/modifier l'etat de classe ou servir de constructeur alternatif. Une `@staticmethod` ne recoit ni `self` ni `cls` : c'est une fonction normale simplement rattachee a la classe par organisation logique, sans acces automatique a l'instance ou a la classe.

```
class Pizza:
    def __init__(self, ingredients):
        self.ingredients = ingredients

    @classmethod
    def margherita(cls):
        # constructeur alternatif
        return cls(["tomate", "mozzarella"])

    @staticmethod
    def temps_cuisson(taille_cm):
        # utilitaire sans lien avec l'instance
        return taille_cm * 0.5
```

JUNIOR Q15. Difference entre *instance method*, *classmethod* et *staticmethod* ?

Reponse. La methode d'instance (par default, premier parametre `self`) agit sur une instance precise et peut lire/modifier son etat. La `classmethod` agit sur la classe elle-meme (`cls`), utile pour les constructeurs alternatifs ou l'etat partage entre instances. La `staticmethod` n'agit ni sur l'un ni sur l'autre : c'est un simple regroupement logique.

MID Q16. Comment fonctionne l'heritage multiple et le MRO (Method Resolution Order) ?

Reponse. Python utilise l'algorithme C3 linearization pour determiner l'ordre dans lequel les classes parentes sont consultees lors de la resolution d'une methode (`ClassName.__mro__` ou `help(ClassName)`). Cela garantit un ordre coherent meme avec des heritages en diamant, et c'est ce qui permet a `super()` de fonctionner correctement en cooperant avec plusieurs classes parentes.

MID Q17. Qu'est-ce que le *duck typing* ?

Reponse. Principe selon lequel le type d'un objet importe moins que les methodes qu'il implemente : « si ca marche comme un canard et cancale comme un canard, c'est un canard ». Python ne verifie pas de type explicite a la compilation : si un objet possede la methode attendue (par ex. `.read()`), il peut etre utilise la ou un fichier est attendu, meme sans heriter d'une classe commune.

Methodes speciales (dunder)

JUNIOR Q18. Difference entre `__init__` et `__new__` ?

Reponse. `__new__` est la methode statique qui *cre*e et renvoie une nouvelle instance (allocation memoire); `__init__` est appelee *apres*, sur l'instance deja creee, pour l'*initialiser* (definir ses attributs). On surcharge `__new__` dans des cas rares : classes immutables (heritage de `tuple`, `str`), singletons, ou metaclasses.

JUNIOR Q19. Difference entre `__str__` et `__repr__` ?

Reponse. `__str__` definit la representation « lisible » destinee a l'utilisateur final (appelee par `print()` et `str()`). `__repr__` definit une representation « non ambigue » destinee au developpeur/debug (appelee dans le REPL, par `repr()`, et en secours si `__str__` n'est pas defini). Regle courante : `__repr__` devrait, si possible, ressembler a du code Python valide permettant de recreer l'objet.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Point(x={self.x}, y={self.y})"
    def __str__(self):
        return f"({self.x}, {self.y})"
```

MID Q20. Cite d'autres dunder methods utiles et leur role.

Reponse. `__len__` (pour `len(obj)`), `__eq__`/`__lt__` (comparaisons), `__iter__`/`__next__` (iteration), `__getitem__`/`__setitem__` (indexation `obj[i]`), `__enter__`/`__exit__` (context manager `with`), `__call__` (rendre l'objet appellable comme une fonction), `__hash__` (utilisation comme cle de dict).

Decorateurs

JUNIOR Q21. Qu'est-ce qu'un decorateur ?

Reponse. Une fonction qui prend une autre fonction (ou classe) en argument et renvoie une version modifiée/enrichie de celle-ci, sans changer son code source. Cela repose sur le fait que les fonctions sont des objets de première classe en Python (on peut les passer en argument, les stocker, les renvoyer).

```
def chrono(fonction):
    def wrapper(*args, **kwargs):
        debut = time.time()
        resultat = fonction(*args, **kwargs)
        print(f"{fonction.__name__} : {time.time() - debut:.3f}s")
        return resultat
    return wrapper

@chrono
def calcul_lourd():
    ...
```

MID Q22. Comment fonctionne @property ?

Reponse. @property transforme une méthode en attribut en lecture (accessible sans parenthèses), permettant d'ajouter de la logique (validation, calcul) tout en gardant une API simple type attribut. Combinée à @x.setter, elle permet de contrôler l'écriture (par ex. valider une valeur avant de l'assigner) sans casser l'interface publique de la classe.

```
class Compte:
    def __init__(self, solde):
        self._solde = solde

    @property
    def solde(self):
        return self._solde

    @solde.setter
    def solde(self, valeur):
        if valeur < 0:
            raise ValueError("Solde negatif interdit")
        self._solde = valeur
```

MID Q23. Comment écrire un decorateur qui accepte des parametres ?

Reponse. Il faut un niveau d'imbrication supplémentaire : une fonction qui prend les paramètres du decorateur et renvoie le « vrai » decorateur.

```
def repeter(n):
    def decorateur(fonction):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                resultat = fonction(*args, **kwargs)
            return resultat
        return wrapper
    return decorateur

@repeter(3)
```



```
def dire_bonjour():  
    print("Bonjour")
```

SENIOR Q24. Pourquoi utiliser `functools.wraps` dans un decorateur ?

Reponse. Sans `@functools.wraps(fonction)` sur le `wrapper`, la fonction decoree perd ses metadonnees d'origine (`__name__`, `__doc__`, signature), ce qui casse l'introspection, la documentation automatique et certains outils (debogueurs, frameworks comme FastAPI qui inspectent la signature pour l'injection de dependances).

Generateurs et iterateurs

JUNIOR Q25. Difference entre un *iterable* et un *iterateur* ?

Reponse. Un *iterable* est un objet capable de produire un iterateur (il implemente `__iter__`), comme une liste. Un *iterateur* est l'objet qui garde l'etat de progression et produit les valeurs une a une via `__next__`, jusqu'a lever `StopIteration`. Une liste est iterable mais n'est pas elle-meme un iterateur : `iter(liste)` en produit un nouveau a chaque appel.

MID Q26. A quoi sert `yield` et comment fonctionne un generateur ?

Reponse. `yield` transforme une fonction en *generateur* : au lieu de calculer et retourner tout le resultat d'un coup, la fonction produit les valeurs une par une et *suspend son execution* entre chaque appel, conservant son etat local. Avantage majeur : la memoire consommee reste constante meme sur une sequence tres grande (voire infinie), car rien n'est stocke en liste intermediaire.

```
def carres(n):  
    for i in range(n):  
        yield i * i  
  
for c in carres(1_000_000):    # pas de liste de 1M elements en memoire  
    ...
```

MID Q27. Quand preferer un generateur a une liste ?

Reponse. Des que la sequence est grande ou potentiellement infinie, ou que l'appelant n'a pas besoin de tous les elements en meme temps (traitement en flux, pipeline). La contrepartie : un generateur ne se parcourt qu'une seule fois et n'offre pas d'accès aleatoire (`gen[3]` impossible) ni `len()`.

Gestion memoire

MID Q28. Comment fonctionne le *reference counting* en CPython ?

Reponse. Chaque objet garde un compteur du nombre de references qui pointent vers lui. Quand ce compteur tombe a zero (plus personne ne le reference), l'objet est immediatement desalloue. C'est le mecanisme principal de gestion memoire de CPython, complete par un *garbage collector* pour les cas que le comptage seul ne resout pas.

MID Q29. Pourquoi a-t-on besoin d'un Garbage Collector en plus du reference counting ?

Reponse. Le comptage de references seul ne detecte pas les *cycles de reference* (deux objets qui se referencent mutuellement, meme si plus rien d'exterieur ne les reference – leur compteur ne retombe jamais a zero). Le module `gc` implemente un algorithme de detection de cycles (generational garbage collection) qui parcourt periodiquement les objets pour liberer ces cycles orphelins.

MID Q30. Qu'est-ce que la regle LEGB ?

Reponse. Ordre de resolution des noms de variables en Python : **L**ocal (dans la fonction courante) → **E**nclosing (fonction englobante, pour les closures) → **G**lobal (module) → **B**uilt-in (fonctions natives comme `len`, `print`). Python cherche le nom dans cet ordre et s'arrete a la premiere correspondance.

SENIOR Q31. Qu'est-ce que le GIL (Global Interpreter Lock) et quel est son impact ?

Reponse. Un verrou global qui n'autorise qu'un seul thread a executer du bytecode Python a la fois dans un processus CPython, meme sur une machine multi-coeurs. Consquence : le multithreading n'accelere pas les taches *CPU-bound* (calcul pur), mais reste efficace pour les taches *I/O-bound* (le GIL est relache pendant les attentes reseau/disque). Pour paralleliser du calcul, on utilise `multiprocessing` (processus separes, chacun avec son propre GIL) plutot que `threading`.

Exceptions

JUNIOR Q32. Explique le fonctionnement de `try / except / else / finally`.

Reponse. Le bloc `try` contient le code a risque. `except` capture et traite une exception specifique. `else` s'execute uniquement si *aucune* exception n'a ete levee. `finally` s'execute *toujours*, qu'il y ait eu exception ou non (utile pour liberer des ressources : fermer un fichier, une connexion).

```
try:
    valeur = int(entree)
except ValueError:
    print("Entree invalide")
else:
    print(f"Conversion reussie : {valeur}")
finally:
    print("Nettoyage effectue")
```

MID Q33. Comment creer et utiliser une exception personnalisee ?

Reponse. On herite de `Exception` (ou d'une sous-classe plus specifique) pour creer un type d'erreur metier explicite, plus facile a capturer et documenter qu'une exception generique.

```
class SoldeInsuffisantError(Exception):
    def __init__(self, solde, montant):
        self.solde = solde
        self.montant = montant
        super().__init__(
            f"Solde {solde} insuffisant pour retirer {montant}")

def retirer(solde, montant):
    if montant > solde:
        raise SoldeInsuffisantError(solde, montant)
```

MID Q34. Quelle difference entre `raise Erreur()` et `raise Erreur() from autre_erreur`?

Reponse. `raise ... from ...` chaine explicitement une nouvelle exception a sa cause d'origine, en preservant la traceback complete dans l'attribut `__cause__`. Tres utile pour transformer une erreur bas niveau (ex. `KeyError`) en une erreur metier explicite sans perdre le contexte de debogage.

Async Python

MID Q35. Que signifie `async/await` et qu'est-ce qu'une coroutine ?

Reponse. `async def` definit une *coroutine* : une fonction dont l'execution peut etre suspendue et reprise plus tard sans bloquer le thread. `await` suspend la coroutine courante jusqu'a ce que l'operation attendue (typiquement une I/O) se termine, en rendant la main a l'*event loop* pour qu'il execute d'autres taches pendant l'attente.

MID Q36. Qu'est-ce que l'*event loop* dans `asyncio` ?

Reponse. Le coeur du modele asynchrone : une boucle qui gere un ensemble de taches (coroutines) et bascule de l'une a l'autre chaque fois qu'une tache `await` une operation I/O, au lieu de rester bloquee a attendre. Contrairement au multithreading, tout se passe sur **un seul thread** : c'est de la concurrence cooperative, pas du parallelisme.

MID Q37. Pourquoi `async` peut rendre une application plus rapide alors que Python reste mono-thread (a cause du GIL) ?

Reponse. Le gain ne vient pas de calcul parallele mais du fait de ne plus *bloquer* le thread pendant les attentes I/O (requetes reseau, base de donnees, disque). Pendant qu'une requete attend une reponse externe, l'*event loop* peut traiter d'autres requetes en parallele logique. Pour une API qui passe le plus clair de son temps a attendre la base de donnees ou un service externe, cela permet de servir beaucoup plus de requetes concurrentes avec les memes ressources qu'en mode synchrone bloquant.

SENIOR Q38. Quel est le piege classique du code async : melanger appel bloquant et async ?

Reponse. Appeler une fonction *synchrone bloquante* (ex. `requests.get()`, un calcul CPU lourd, un driver DB non-async) depuis une coroutine bloque tout l'*event loop* – toutes les autres taches concurrentes sont geles pendant ce temps, annulant le benefice de l'asynchrone. Il faut soit utiliser une librairie async-native (`httpx`, `asyncpg`), soit deporter l'appel bloquant dans un threadpool (`run_in_executor`, ou `fastapi's run_in_threadpool`).

Piege frequent

Un `def` classique appele sans `await` dans un contexte async n'est **pas** execute en parallele : il bloque comme du code synchrone normal.

3. Django

Architecture

JUNIOR Q39. Explique le modele MTV de Django.

Reponse. Model–Template–View, variante du MVC. Le **Model** definit la structure des donnees et la logique d'accès (ORM). Le **Template** gere le rendu HTML (presentation). La **View** contient la logique metier : elle recoit la requete, interroge les Models, choisit et alimente le Template. Django lui meme joue le role du « Controller » du MVC classique (routage, dispatch).

MID Q40. Que se passe-t-il lorsqu'une requete HTTP arrive dans Django ?

Reponse. 1) Le serveur WSGI/ASGI recoit la requete et la transmet a Django. 2) Les **middlewares** la traitent dans l'ordre (auth, session, CSRF...). 3) Le **resolveur d'URL** (`urls.py`) trouve la vue correspondante. 4) La **vue** s'exécute : elle interroge les **models** via l'ORM, applique la logique metier. 5) La vue renvoie une **HttpResponse** (souvent generee via un **template**, ou du JSON avec DRF). 6) La reponse retrace les middlewares (dans l'ordre inverse) avant d'être renvoyee au client.

Routing, vues, middleware

JUNIOR Q41. A quoi servent `urls.py`, `path()` et `include()` ?

Reponse. `urls.py` declare la table de routage de l'application : association entre motifs d'URL et vues. `path()` definit une route unique (avec parametres typables, ex. `<int:id>`). `include()` permet de deleguer un prefixe d'URL vers le `urls.py` d'une autre application, favorisant la modularite (chaque app Django gere ses propres routes).

JUNIOR Q42. Function Based View vs Class Based View : pourquoi utiliser une CBV ?

Reponse. Une FBV est une simple fonction qui recoit la requete ; une CBV encapsule le comportement dans une classe (souvent en heritant de vues generiques comme `ListView`, `CreateView`). Les CBV favorisent la reutilisation via l'heritage et les mixins, reduisent le code repetitif pour les patterns CRUD standards, et separent proprement les methodes par verbe HTTP (`get()`, `post()`...). Les FBV restent plus lisibles pour une logique simple ou tres specifique.

MID Q43. Qu'est-ce qu'un middleware Django ?

Reponse. Un composant qui s'insere dans le traitement de *chaque* requete/reponse, sous forme de couches successives (pipeline). Utilise pour des preoccupations transverses : authentication de session, gestion CSRF, compression, logging, gestion des CORS, limitation de debit. Chaque middleware peut modifier la requete avant qu'elle atteigne la vue, ou la reponse avant qu'elle reparte vers le client.

```
class TempsReponseMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        debut = time.time()
        response = self.get_response(request)
        response["X-Temps-Reponse"] = f"{time.time() - debut:.3f}"
        return response
```

ORM

JUNIOR Q44. Difference entre `filter()` et `get()` ?

Reponse. `filter()` renvoie toujours un `QuerySet` (potentiellement vide, avec 0, 1 ou plusieurs resultats), evalue de maniere paresseuse. `get()` renvoie *un seul objet* et leve `DoesNotExist` si aucun resultat, ou `MultipleObjectsReturned` si plusieurs correspondent – a utiliser uniquement quand on est certain qu’au plus un enregistrement correspond (ex. recherche par cle primaire).

JUNIOR Q45. Que fait `exclude()` ?

Reponse. L’inverse de `filter()` : renvoie un `QuerySet` contenant les objets qui *ne* correspondent *pas* a la condition donnee.

MID Q46. Que sont `annotate()` et `aggregate()` ?

Reponse. `aggregate()` calcule une valeur agregee sur l’ensemble du queryset (ex. `Avg`, `Count`, `Sum`) et renvoie un dictionnaire unique. `annotate()` calcule une valeur agregee *par ligne* (souvent apres un `group by` implicite), en ajoutant un champ calcule a chaque objet du queryset – par exemple, le nombre de commandes par client.

```
from django.db.models import Count, Avg

Client.objects.annotate(nb_commandes=Count("commandes"))
Commande.objects.aggregate(total=Avg("montant"))
```

MID Q47. Que sont `select_related()` et `prefetch_related()` ? Pourquoi sont-ils critiques pour la performance ?

Reponse. Ce sont des optimisations pour eviter le probleme des **requetes N+1**. `select_related()` effectue une jointure SQL (`JOIN`) pour recuperer en une seule requete les objets lies via `ForeignKey` ou `OneToOneField` (relations « vers un seul »). `prefetch_related()` effectue une requete SQL *separee* supplementaire puis fait la jointure en Python – necessaire pour les relations `ManyToManyField` ou les relations inverses (« vers plusieurs »), ou un simple `JOIN SQL` ne suffit pas.

```
# N+1 : 1 requete + 1 requete par article pour son auteur
articles = Article.objects.all()
for a in articles:
    print(a.auteur.nom)           # requete SQL a chaque iteration !

# Corrige : 1 seule requete grace au JOIN
articles = Article.objects.select_related("auteur")
```

MID Q48. Qu’est-ce que le probleme N+1 et comment le detecter ?

Reponse. Il survient quand on execute 1 requete pour recuperer une liste d’objets, puis 1 requete *supplementaire par objet* pour recuperer une donnee liee (ex. l’auteur de chaque article dans une boucle) – soit $N + 1$ requetes au lieu d’une seule optimisee. On le detecte via `django-debug-toolbar` (compteur de requetes SQL), les logs SQL en developpement, ou des outils d’APM en production (New Relic, Datadog). Se corrige avec `select_related/prefetch_related`.

Relations, migrations, admin, signals, forms

JUNIOR Q49. Difference entre `ForeignKey`, `OneToOneField` et `ManyToManyField` ?

Reponse. `ForeignKey` : relation plusieurs-vers-un (plusieurs commandes pour un client). `OneToOneField` : relation un-vers-un stricte (un utilisateur et son profil etendu). `ManyToManyField` : relation plusieurs-vers-plusieurs, implementee via une table intermediaire (ex. articles et tags).

JUNIOR Q50. Pourquoi `makemigrations` et `migrate` sont-ils deux commandes distinctes ?

Reponse. `makemigrations` *genere* les fichiers de migration en comparant l'etat des models a la derniere migration connue (c'est une etape locale, relisable et versionnable dans git). `migrate` *applique* reellement ces migrations a la base de donnees. Separer les deux permet de relire/modifier une migration avant de l'executer, et de deployer la meme migration de maniere identique sur plusieurs environnements (dev, staging, prod) sans la regenerer a chaque fois.

JUNIOR Q51. Pourquoi Django Admin existe-t-il ?

Reponse. Interface d'administration generee automatiquement a partir des models, permettant de gerer les donnees (CRUD) sans ecrire de vues ni de templates. Tres utile en developpement et pour donner un back-office rapide a des utilisateurs non techniques, mais generalement pas destine a etre expose publiquement tel quel en production sans durcissement (permissions, 2FA, IP whitelisting).

MID Q52. Qu'est-ce qu'un signal Django (`post_save`, `pre_save`) et quand l'utiliser ?

Reponse. Un signal permet a du code decouple de reagir a un evenement du cycle de vie d'un model (avant/apres sauvegarde, suppression...) sans modifier directement la methode `save()`. Exemple : creer automatiquement un `Profile` a chaque creation d'`User` via `post_save`. A utiliser avec parcimonie : les signaux rendent le flux d'execution moins explicite (« action a distance ») et peuvent compliquer le debogage – souvent, surcharger `save()` ou utiliser un service explicite est plus lisible.

MID Q53. A quoi sert `ModelForm` et la methode `clean()` ?

Reponse. `ModelForm` genere automatiquement un formulaire (champs, widgets, validations de base) a partir d'un `Model`, evitant la duplication entre schema de donnees et schema de formulaire. La methode `clean()` (ou `clean_<champ>()`) permet d'ajouter de la validation personnalisee, executee apres les validations de champ individuelles, notamment pour verifier la coherence entre plusieurs champs.

4. Django REST Framework (DRF)

JUNIOR Q54. Pourquoi utiliser DRF plutôt que Django seul pour une API ?

Reponse. Django seul ne fournit pas nativement les briques attendues d'une API REST moderne : serialisation/deserialisation JSON propre, negociation de contenu, authentication par token, pagination, permissions granulaires, documentation OpenAPI. DRF fournit ces briques de maniere standardisee et extensible, tout en restant integre a l'ORM et aux permissions Django existantes.

JUNIOR Q55. Difference entre `Serializer` et `ModelSerializer` ?

Reponse. `Serializer` se definit champ par champ manuellement (controle total, utile pour des structures qui ne correspondent a aucun model). Un `ModelSerializer` genere automatiquement les champs et validateurs a partir d'un `Model` Django (comme `ModelForm` pour les formulaires), reduisant fortement le code repetitif pour les cas standards.

MID Q56. Difference entre `APIView`, `GenericAPIView` et `ViewSet` ?

Reponse. `APIView` est le niveau le plus bas : on ecrit soi-meme `get()/post()`, controle total mais plus verbeux. `GenericAPIView` ajoute des comportements standards (`queryset`, `serializer_class`, pagination) combinables avec des *mixins* (`ListModelMixin`, `CreateModelMixin`...) pour construire du CRUD avec peu de code. `ViewSet` regroupe la logique CRUD complete d'une ressource dans une seule classe (`list`, `retrieve`, `create`, `update`, `destroy`), pensee pour etre branchee a un `Router` qui genere automatiquement les URLs correspondantes.

MID Q57. Role d'un `Router` en DRF ?

Reponse. Genere automatiquement les routes URL standards pour un `ViewSet` (ex. `GET /users/`, `POST /users/`, `GET /users/{id}/...`) sans avoir a les declarer manuellement dans `urls.py`, en suivant les conventions REST.

JUNIOR Q58. Comment fonctionnent `Permission` et `Authentication` en DRF ?

Reponse. **Authentication** determine *qui* fait la requete (identification : session, token, JWT...). **Permission** determine *ce que l'utilisateur identifie a le droit de faire* (autorisation : lecture seule, propriétaire uniquement, admin...). Les deux s'executent avant que la vue ne traite la requete, et peuvent etre combinees/personnalisees par vue ou globalement.

MID Q59. Pourquoi la pagination est-elle importante sur une API qui liste des ressources ?

Reponse. Sans pagination, un endpoint de liste renverrait potentiellement des millions d'enregistrements en une seule reponse, saturant la memoire serveur, le reseau et le client. La pagination (par page, par curseur, ou par offset/limit) borne la taille de chaque reponse et permet au client de parcourir les resultats progressivement. DRF fournit `PageNumberPagination`, `LimitOffsetPagination` et `CursorPagination` (plus stable sur des donnees qui changent).

MID Q60. Difference entre `filtering` et `ordering` dans une API DRF ?

Reponse. Le *filtering* restreint les resultats selon des criteres (`?statut=actif`), tandis que l'*ordering* controle l'ordre de tri des resultats (`?ordering=-date_creation`). Les deux sont souvent geres via des parametres de query string et des backends dedies (`django-filter`, `OrderingFilter`) plutot que codes en dur dans chaque vue.

5. FastAPI

JUNIOR Q61. Qu'est-ce que FastAPI et pourquoi est-il reconnu pour sa rapidité ?

Reponse. Un framework web Python moderne, basé sur **Starlette** (couche ASGI) et **Pydantic** (validation de données), conçu nativement pour l'asynchrone. Sa rapidité d'exécution vient de son fondement ASGI/async (haute concurrence I/O) et de Pydantic v2 (validation écrite en Rust). Sa rapidité de *developpement* vient de la génération automatique de documentation OpenAPI et de la validation automatique des types via les annotations Python.

JUNIOR Q62. Quelle est la différence entre WSGI et ASGI ?

Reponse. WSGI (*Web Server Gateway Interface*) est synchrone : chaque requête occupe un worker jusqu'à sa fin, ce qui est inefficace pour les opérations longues (I/O). ASGI (*Asynchronous Server Gateway Interface*) supporte nativement l'asynchrone, le WebSocket et les connexions longue durée, permettant à un seul worker de gérer de nombreuses requêtes concurrentes pendant qu'elles attendent des I/O. Django classique tourne en WSGI (Gunicorn), FastAPI tourne en ASGI (Uvicorn).

MID Q63. Pourquoi définir une route avec `async def` plutôt que `def` simple dans FastAPI ?

Reponse. Avec `async def`, la fonction s'exécute directement dans l'évent loop : idéal si tout le corps de la fonction est non-bloquant (appels à des bibliothèques async comme `asyncpg`, `httpx`). Avec un simple `def`, FastAPI exécute automatiquement la fonction dans un *threadpool* séparé pour ne pas bloquer l'évent loop – utile si le code appelle des bibliothèques synchrones bloquantes. Le piège est d'utiliser `async def` tout en appelant du code bloquant à l'intérieur : cela gèle l'évent loop entier.

Pydantic

JUNIOR Q64. À quoi sert `BaseModel` de Pydantic ?

Reponse. Classe de base pour définir des schémas de données types : Pydantic valide automatiquement les types (et lève une erreur claire sinon), convertit/parse les données brutes (JSON, dict) vers des objets Python types, et permet la sérialisation inverse (objet → dict/JSON). C'est le mécanisme qui alimente la validation automatique des requêtes et la génération du schéma OpenAPI dans FastAPI.

```
from pydantic import BaseModel

class UtilisateurCreate(BaseModel):
    nom: str
    email: str
    age: int | None = None
```

MID Q65. Différence entre validation et sérialisation dans Pydantic ?

Reponse. La **validation** (parsing) convertit et vérifie des données brutes entrantes (dict, JSON) vers une instance typée du modèle, en levant une erreur détaillée si un champ est invalide ou manquant. La **sérialisation** fait l'opération inverse : transformer l'instance du modèle en dict/JSON pour la renvoyer au client, avec un contrôle fin (exclusion de champs, alias, mode JSON vs Python).

Dependances, requetes, reponses

JUNIOR Q66. Que fait `Depends()` et pourquoi l'utiliser ?

Reponse. Mecanisme d'*injection de dependances* de FastAPI : on declare une fonction (ou classe) qui produit une valeur (connexion DB, utilisateur authentifie, configuration) et FastAPI l'execute automatiquement avant la route, en injectant le resultat comme parametre. Cela permet de **reutiliser** de la logique commune (auth, pagination, session DB) entre plusieurs endpoints sans duplication, tout en restant facilement testable (on peut surcharger une dependance dans les tests).

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users/{id}")
def get_user(id: int, db: Session = Depends(get_db)):
    return db.query(User).get(id)
```

JUNIOR Q67. Comment recuperer Path Parameters, Query Parameters, Headers, Cookies et Body dans FastAPI ?

Reponse. Il suffit d'annoter les parametres de la fonction de route : un parametre present dans le chemin de l'URL est un *path parameter* automatique ; un parametre simple non present dans le chemin devient un *query parameter* ; `Header()`, `Cookie()` et `Body()` (ou un `BaseModel`) rendent explicite la source des autres donnees. FastAPI deduit la source depuis la signature de la fonction et le type declare.

```
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None,
              x_token: str = Header(...)):
    ...
```

MID Q68. Quelles sont les principales classes de reponse (`Response`) en FastAPI ?

Reponse. `JSONResponse` (reponse JSON explicite, utile pour un code de statut ou des headers personnalisés), `Response` (reponse brute generique), `StreamingResponse` (envoi progressif de donnees, utile pour de gros fichiers ou du streaming de reponse LLM), `FileResponse` (envoi efficace d'un fichier depuis le disque, avec support des headers de cache et range requests).

MID Q69. Comment ajouter un middleware dans FastAPI ?

Reponse. Via le decorateur `@app.middleware("http")` ou `app.add_middleware(...)`. Un middleware HTTP recoit la requete, peut la modifier ou executer du code avant, appelle `await call_next(request)` pour continuer le traitement, puis peut modifier la reponse avant de la renvoyer.

```
@app.middleware("http")
async def ajouter_temps_traitement(request, call_next):
    debut = time.time()
    response = await call_next(request)
    response.headers["X-Temps"] = str(time.time() - debut)
    return response
```

MID Q70. Pourquoi utiliser `BackgroundTasks` ?

Reponse. Pour executer une operation *apres* avoir renvoye la reponse au client, sans le faire attendre (ex. envoi d'un email de confirmation, ecriture de log, notification). Contrairement a une vraie file de taches (Celery, RQ), `BackgroundTasks` s'execute dans le meme processus juste apres la reponse : adapte a des taches courtes et non critiques, pas a des traitements lourds ou qui necessitent des garanties de fiabilite (retry, persistance).

MID Q71. A quoi servent les *lifespan events* (`startup/shutdown`) ?

Reponse. Permettent d'executer du code une seule fois au demarrage de l'application (ex. ouvrir un pool de connexions DB, charger un modele ML en memoire) et au moment de l'arret (fermer proprement les connexions, liberer les ressources). Depuis FastAPI recent, on utilise le context manager `lifespan` plutot que les evenements `@app.on_event` (deprecies).

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await connect_to_db()      # startup
    yield
    await disconnect_from_db() # shutdown

app = FastAPI(lifespan=lifespan)
```

JUNIOR Q72. Pourquoi la documentation Swagger/OpenAPI est-elle generee automatiquement par FastAPI ?

Reponse. Parce que FastAPI construit le schema OpenAPI directement a partir des *annotations de type* des fonctions de route et des modeles Pydantic – il n'y a pas de documentation a maintenir separement du code : le schema (donc la doc `/docs` via Swagger UI et `/redoc`) reste automatiquement synchronise avec l'implementation reelle.

6. REST API

JUNIOR Q73. Rappelle le role des principaux verbes HTTP.

Reponse. GET : lire une ressource (sans effet de bord). POST : creer une nouvelle ressource / declencher une action. PUT : remplacer integralement une ressource existante. PATCH : modifier partiellement une ressource. DELETE : supprimer une ressource.

JUNIOR Q74. Quelle difference entre PUT et PATCH ?

Reponse. PUT remplace la ressource *entiere* : le client doit envoyer toutes les donnees, les champs omis sont generalement reinitialises. PATCH applique une modification *partielle* : seuls les champs envoyes sont mis a jour, les autres restent inchanges.

MID Q75. Qu'est-ce que l'idempotence en REST ? Quelles methodes sont idempotentes ?

Reponse. Une operation est idempotente si l'executer plusieurs fois produit le meme resultat/etat final que l'executer une seule fois (le client peut donc la reessayer sans risque apres un timeout). GET, PUT, DELETE sont idempotentes ; POST ne l'est generalement pas (appeler deux fois POST /commandes cree deux commandes). Notion importante pour concevoir des retries reseau surs.

JUNIOR Q76. Explique les codes de statut HTTP les plus courants.

Reponse. **2xx (succes)** : 200 OK, 201 Created (apres un POST reussi), 204 No Content (succes sans corps de reponse, souvent apres DELETE). **4xx (erreur client)** : 400 Bad Request (requete malformee), 401 Unauthorized (non authentifie), 403 Forbidden (authentifie mais non autorise), 404 Not Found, 409 Conflict (etat incompatible, ex. doublon), 422 Unprocessable Entity (validation echouee – tres frequent avec FastAPI/Pydantic). **5xx (erreur serveur)** : 500 Internal Server Error.

7. SQL

JUNIOR Q77. Difference entre INNER JOIN et LEFT JOIN ?

Reponse. INNER JOIN ne renvoie que les lignes ayant une correspondance dans *les deux* tables. LEFT JOIN renvoie toutes les lignes de la table de gauche, meme sans correspondance a droite (les colonnes de droite sont alors NULL) – utile pour lister par exemple tous les clients, y compris ceux sans commande.

JUNIOR Q78. Difference entre WHERE et HAVING (avec GROUP BY) ?

Reponse. WHERE filtre les lignes *avant* l'agregation (GROUP BY), ligne par ligne, et ne peut pas référencer une fonction d'agregation. HAVING filtre les groupes *apres* l'agregation, et peut donc utiliser des fonctions comme COUNT(), SUM().

```
SELECT client_id, COUNT(*) AS nb
FROM commandes
WHERE statut = 'validee'
GROUP BY client_id
HAVING COUNT(*) > 5;
```

MID Q79. A quoi sert un INDEX et quel est son cout ?

Reponse. Un index accelere les recherches/filtres/tris (WHERE, ORDER BY, JOIN) sur une colonne, en evitant un scan complet de la table (structure typiquement en B-tree, en $O(\log n)$ plutot que $O(n)$). Le cout : chaque INSERT/UPDATE/DELETE doit aussi mettre a jour l'index, ce qui ralentit les ecritures et consomme de l'espace disque supplementaire – il faut donc indexer les colonnes frequemment filterees, pas toutes les colonnes.

JUNIOR Q80. Difference entre PRIMARY KEY, FOREIGN KEY et UNIQUE ?

Reponse. PRIMARY KEY identifie de maniere unique chaque ligne d'une table (implique unicite + non-nullite, indexee automatiquement). FOREIGN KEY garantit l'integrite referentielle en liant une colonne a la cle primaire d'une autre table. UNIQUE garantit qu'une colonne (ou combinaison de colonnes) ne contient pas de doublon, sans en faire l'identifiant de la ligne.

MID Q81. Qu'est-ce qu'une transaction SQL ?

Reponse. Un ensemble d'operations executees comme une unite atomique : soit toutes reussissent (COMMIT), soit aucune n'est appliquee (ROLLBACK en cas d'erreur). Essentiel pour garantir la coherence des donnees lors d'operations multi-etapes (ex. debiter un compte et en crediter un autre doivent reussir ensemble ou pas du tout).

MID Q82. Exercice – Ecris une requete qui recupere les 10 derniers utilisateurs inscrits.

Reponse.

```
SELECT *
FROM utilisateurs
ORDER BY date_inscription DESC
LIMIT 10;
```

MID Q83. Exercice – Compte le nombre d'utilisateurs par pays, tries par nombre decroissant.

Reponse.

```
SELECT pays, COUNT(*) AS nb_utilisateurs
FROM utilisateurs
GROUP BY pays
ORDER BY nb_utilisateurs DESC;
```

8. PostgreSQL

MID Q84. Qu'est-ce que ACID ?

Reponse. Quatre garanties attendues d'une transaction fiable : **Atomicite** (tout ou rien), **Coherence** (la base passe d'un etat valide a un autre etat valide, respecte les contraintes), **Isolation** (des transactions concurrentes ne s'interferent pas de maniere incoherente), **Durabilite** (une fois validee, une transaction survit meme a une panne).

SENIOR Q85. Que sont les niveaux d'isolation (Isolation Level) ?

Reponse. Ils definissent a quel point une transaction est protegee des effets des autres transactions concurrentes, du plus permissif au plus strict : `READ UNCOMMITTED`, `READ COMMITTED` (default PostgreSQL), `REPEATABLE READ`, `SERIALIZABLE`. Plus le niveau est strict, plus les anomalies (dirty read, non-repeatable read, phantom read) sont evitees, mais plus le risque de contention/rollback augmente.

SENIOR Q86. Qu'est-ce qu'un deadlock et comment l'eviter ?

Reponse. Situation ou deux (ou plus) transactions s'attendent mutuellement pour liberer des verrous qu'elles detiennent, bloquant indefiniment – PostgreSQL detecte ces cycles et annule automatiquement une des transactions (erreur `deadlock detected`). Pour limiter le risque : toujours acquerir les verrous/mettre a jour les lignes dans le *memme ordre* dans toute l'application, garder les transactions courtes, eviter les transactions interactives qui attendent une action utilisateur en cours de route.

9. Securite

JUNIOR Q87. Difference entre Authentication et Authorization ?

Reponse. **Authentication** (authentification) repond a « qui es-tu ? » (verifier l'identite : mot de passe, token). **Authorization** (autorisation) repond a « as-tu le droit de faire ceci ? » (verifier les permissions une fois l'identite etablie).

MID Q88. Qu'est-ce qu'un JWT et comment fonctionne-t-il ?

Reponse. JSON Web Token : un jeton auto-suffisant compose de trois parties encodees en base64 et separees par des points – *header* (algorithme), *payload* (donnees/claims, ex. id utilisateur, expiration), *signature* (garantit l'integrite, calculee avec une cle secrete ou une paire de cles). Le serveur peut verifier la validite d'un JWT sans requete base de donnees (juste verifier la signature), ce qui le rend adapte aux architectures distribuees/stateless.

Piege frequent

Un JWT n'est pas chiffre par default (juste signe) : ne jamais y mettre de donnees sensibles en clair, tout le monde peut decoder le payload.

MID Q89. Difference entre Access Token et Refresh Token ?

Reponse. L'**access token** a une duree de vie courte et est envoye a chaque requete pour prouver l'identite – s'il est vole, l'exposition est limitee dans le temps. Le **refresh token** a une duree de vie plus longue et sert uniquement a obtenir un nouvel access token sans redemander les identifiants, generalement stocke de maniere plus securisee (cookie httpOnly) et revocable cote serveur.

MID Q90. Explique le principe general d'OAuth2.

Reponse. Protocole d'autorisation delegue : un utilisateur autorise une application tierce a acceder a ses ressources (sur un autre service) sans lui partager son mot de passe. L'application recoit un *access token* avec un scope limite, delivre par un serveur d'autorisation apres consentement explicite de l'utilisateur. FastAPI et DRF fournissent tous deux des integrations pour les flows OAuth2 standards (ex. *password flow*, *authorization code*).

JUNIOR Q91. Pourquoi ne jamais stocker un mot de passe en clair ? Role de bcrypt.

Reponse. Si la base de donnees fuite, des mots de passe en clair exposent immediatement tous les comptes (et souvent d'autres services, via la reutilisation de mots de passe). On stocke a la place un *hash* irreversible du mot de passe. **bcrypt** (comme **argon2**) est concu pour etre volontairement *lent* et inclut un *salt* (valeur aleatoire) unique par mot de passe, ce qui rend les attaques par force brute et les rainbow tables couteuses, contrairement a un hash rapide generique comme MD5 ou SHA-256 seul.

MID Q92. Qu'est-ce que CORS et pourquoi le navigateur le bloque-t-il par default ?

Reponse. *Cross-Origin Resource Sharing* : mecanisme de securite du navigateur qui bloque par default les requetes JavaScript vers un domaine different de celui qui a servi la page (*same-origin policy*), pour empecher un site malveillant de faire des requetes authentifiees vers un autre service au nom de l'utilisateur. Le serveur doit explicitement autoriser les origines souhaitees via des en-tetes **Access-Control-Allow-Origin**.

MID **Q93.** Qu'est-ce que CSRF et comment Django s'en protege-t-il ?

Reponse. *Cross-Site Request Forgery* : une attaque ou un site malveillant fait executer, a l'insu de l'utilisateur, une requete d'etat (ex. changement de mot de passe) vers un site ou il est deja authentifie (via ses cookies de session). Django genere un *token CSRF* unique par session, inclus dans chaque formulaire, et verifie sa presence/validite a chaque requete modifiant l'etat (POST/PUT/DELETE) – un attaquant externe ne peut pas connaitre ce token.

MID **Q94.** Qu'est-ce que XSS et comment s'en proteger ?

Reponse. *Cross-Site Scripting* : injection de code JavaScript malveillant dans une page consultee par d'autres utilisateurs (via un champ de saisie non echappe, par exemple). Protection : toujours *echapper*/encoder les donnees utilisateur avant de les injecter dans le HTML (les moteurs de template Django echappent par default), valider les entrees, et definir une politique **Content-Security-Policy** stricte.

JUNIOR **Q95.** Qu'est-ce qu'une injection SQL et comment l'ORM protege-t-il ?

Reponse. Une injection SQL survient quand une entree utilisateur est concatenee directement dans une requete SQL, permettant a un attaquant d'y injecter du SQL arbitraire (ex. ' OR '1'='1'). Les ORM (Django ORM, SQLAlchemy) utilisent des *requetes parametrees* par default (les valeurs sont envoyees separement de la requete SQL, jamais concatenees textuellement), ce qui neutralise ce risque tant qu'on evite le SQL brut non parametre (**raw()**, **extra()**) avec des entrees utilisateur directes.

10. Tests

JUNIOR Q96. Difference entre `pytest` et `unittest` ?

Reponse. `unittest` est le framework de test integre a la bibliotheque standard, base sur des classes heritant de `TestCase` avec des methodes `assertX`. `pytest` est une librairie tierce plus expressive : simple `assert` natif, systeme de *fixtures* puissant et composable, tests parametrables, et un ecosysteme de plugins riche (`pytest-django`, `pytest-asyncio`, `pytest-cov`). Les deux restent compatibles entre eux.

MID Q97. A quoi sert une *fixture* en `pytest` ?

Reponse. Une fonction reusable qui prepare un etat/une ressource necessaire a un ou plusieurs tests (ex. une connexion DB de test, un utilisateur factice, un client API), injectee automatiquement dans la fonction de test qui la declare en parametre. Permet de factoriser la mise en place/le nettoyage (*setup/teardown*) sans duplication.

```
import pytest

@pytest.fixture
def utilisateur_test(db):
    return User.objects.create(username="alice")

def test_profil(utilisateur_test):
    assert utilisateur_test.username == "alice"
```

MID Q98. Quand et pourquoi utiliser `mock` dans un test ?

Reponse. Pour remplacer une dependance externe (appel reseau, service tiers, horloge systeme, envoi d'email) par un objet controle, afin de rendre le test rapide, deterministe et independant de systemes externes potentiellement instables ou couteux. Le risque : sur-mocker peut faire passer un test qui ne reflete plus le comportement reel du systeme integre.

```
from unittest.mock import patch

@patch("app.services.envoyer_email")
def test_inscription_envoie_email(mock_email):
    inscrire_utilisateur("alice@test.com")
    mock_email.assert_called_once()
```

JUNIOR Q99. A quoi servent `APIClient` (DRF) et `TestClient` (FastAPI) ?

Reponse. Ce sont des clients HTTP factices permettant de tester les endpoints d'une API sans lancer un vrai serveur reseau : ils appellent directement l'application (WSGI/ASGI) en memoire, ce qui rend les tests rapides et isolés. Ils permettent de simuler des requetes authentifiees, de verifier le code de statut, le corps JSON et les en-tetes de la reponse.

```
# FastAPI
from fastapi.testclient import TestClient
client = TestClient(app)
def test_health():
    assert client.get("/health").status_code == 200

# DRF
from rest_framework.test import APIClient
```

```
def test_liste_users():  
    client = APIClient()  
    response = client.get("/api/users/")  
    assert response.status_code == 200
```

11. Docker

JUNIOR Q100. Difference entre une *image* et un *container* Docker ?

Reponse. Une **image** est un modele en lecture seule (couches empilees) decrivant un systeme de fichiers et une configuration d'execution. Un **container** est une *instance en cours d'execution* de cette image, avec sa propre couche modifiable ephemere et son propre espace processus/reseau isole. Une meme image peut donner naissance a plusieurs containers independants.

JUNIOR Q101. A quoi sert un Dockerfile ?

Reponse. Fichier texte decrivant, instruction par instruction, comment construire une image Docker : image de base (FROM), copie de fichiers (COPY), installation de dependances (RUN), variables d'environnement (ENV), commande de demarrage (CMD).

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

MID Q102. A quoi sert un *volume* Docker ?

Reponse. Un mecanisme de persistance de donnees en dehors du cycle de vie ephemere d'un container : les donnees ecrites dans un volume survivent a la suppression du container (contrairement a la couche modifiable interne). Utile pour les donnees de base de donnees, fichiers uploads, ou pour partager du code entre l'hote et le container en developpement.

MID Q103. Pourquoi utiliser Docker Compose ?

Reponse. Pour definir et orchestrer plusieurs containers lies (ex. app + PostgreSQL + Redis) via un seul fichier declaratif (`docker-compose.yml`), avec leur reseau partage, leurs volumes et leur ordre de demarrage, plutot que de lancer chaque container manuellement avec de longues commandes `docker run`.

12. Git

JUNIOR Q104. Difference entre `merge` et `rebase` ?

Reponse. `merge` combine deux branches en creant un *commit de fusion* qui preserve l'historique exact tel qu'il s'est produit (deux lignes qui se rejoignent). `rebase` *rejoue* les commits d'une branche par dessus une autre, produisant un historique lineaire sans commit de fusion, mais en *reecrivant* les hash de commits – a eviter sur une branche deja partagee/pushee, car cela desynchronise les autres collaborateurs.

MID Q105. A quoi sert `git stash` ?

Reponse. Met de cote temporairement les modifications non commitees (working directory + index) pour revenir a un etat propre, sans avoir a commiter du travail inacheve – utile pour changer rapidement de branche. On les recupere ensuite avec `git stash pop` ou `git stash apply`.

MID Q106. A quoi sert `cherry-pick` ?

Reponse. Applique un commit specifique (identifie par son hash) d'une branche sur une autre, sans fusionner tout l'historique des deux branches – utile pour recuperer un correctif precis sans embarquer d'autres changements non souhaitees.

MID Q107. Difference entre `git reset` et `git revert` ?

Reponse. `reset` deplace le pointeur de branche vers un commit anterieur, *reecrivant* l'historique (dangereux sur une branche partagee car les commits « annules » disparaissent de la branche). `revert` cree un *nouveau* commit qui annule les changements d'un commit precedent, sans reecrire l'historique existant – surer pour annuler un changement deja pousse/partage.

13. Architecture logicielle

MID Q108. Rappelle le principe SOLID.

Reponse. Single Responsibility – une classe a une seule raison de changer. Open/Closed – ouvert a l’extension, ferme a la modification. Liskov Substitution – une sous-classe doit pouvoir remplacer sa classe mere sans casser le comportement attendu. Interface Segregation – preferer plusieurs interfaces specifiques a une seule interface generale. Dependency Inversion – dependre d’abstractions, pas d’implementations concretes.

MID Q109. Qu’est-ce que le Repository Pattern et quel probleme resout-il ?

Reponse. Un pattern qui isole la logique d’accès aux données (requêtes DB) derriere une interface dediee, decouplant la logique metier du mecanisme de persistance concret (ORM, requêtes SQL brutes, API externe). Avantages : logique metier plus facile a tester (on peut injecter un faux repository en test) et possibilite de changer de source de données sans toucher a la logique metier.

MID Q110. Qu’est-ce qu’une *Service Layer* et pourquoi la separer des vues ?

Reponse. Une couche intermediaire qui contient la logique metier (regles, orchestration de plusieurs operations) independamment des vues/controleurs HTTP. Sans elle, la logique metier finit eparpillee dans les vues, difficile a reutiliser (ex. entre une route API et une tache planifiee) et a tester unitairement sans monter tout le framework web.

MID Q111. Qu’est-ce que l’injection de dependances (Dependency Injection) et pourquoi FastAPI en fait un usage central ?

Reponse. Principe consistant a *fournir* les dependances d’un composant depuis l’exterieur (au lieu que le composant les construise lui-meme), ce qui decouple le code et facilite les tests (substitution par un mock/faux objet). `Depends()` de FastAPI est un systeme d’injection de dependances natif : sessions DB, utilisateur authentifie, configuration – tout peut etre injecte de maniere declarative et surchargee facilement en test via `app.dependency_overrides`.

14. Performance

MID **Q112.** Comment reduire l'impact du probleme N+1 au-dela de `select_related/prefetch_related` ?

Reponse. En surveillant le nombre de requetes SQL par endpoint en continu (APM, `django-debug-toolbar` en dev), en ecrivant des tests qui verifient un nombre de requetes borne (`assertNumQueries`), et en revisant les serializers DRF/Pydantic qui accedent souvent a des relations imbriquees sans prefetch explicite.

MID **Q113.** Comment fonctionne le cache avec Redis dans une API Python ?

Reponse. Redis est une base cle-valeur en memoire, tres rapide, utilisee pour stocker temporairement le resultat d'un calcul ou d'une requete couteuse (cache de requete, cache de session, rate limiting, file de taches avec Celery). Le pattern classique : verifier si la cle existe dans Redis (*cache hit*), sinon calculer/interroger la DB puis stocker le resultat dans Redis avec une *expiration* (TTL) avant de le renvoyer.

MID **Q114.** Difference entre pagination et *lazy loading* ?

Reponse. La **pagination** decoupe explicitement les resultats en pages demandees par le client (parametre `page/limit`). Le **lazy loading** (chargement paresseux) retarde le chargement d'une donnee jusqu'a ce qu'elle soit reellement necessaire (ex. un `QuerySet` Django n'execute la requete SQL qu'au moment ou on itere dessus, pas a sa declaration) – les deux limitent le travail effectue en avance, mais a des niveaux differents.

15. Deploiement

JUNIOR Q115. Difference entre Gunicorn et Uvicorn ? Pourquoi parfois les deux ensemble ?

Reponse. **Gunicorn** est un serveur d'application WSGI, gerant un pool de processus *workers* (robustesse, gestion des crashes, redemarrage). **Uvicorn** est un serveur ASGI performant capable de gerer l'asynchrone. En production, on utilise souvent Gunicorn comme *process manager* (gestion des workers, redemarrage) avec des workers de type **UvicornWorker**, combinant la robustesse operationnelle de Gunicorn et les capacites async d'Uvicorn.

MID Q116. Pourquoi placer Nginx devant une application Django/FastAPI ?

Reponse. Nginx sert de *reverse proxy* : il termine les connexions TLS/HTTPS, sert les fichiers statiques efficacement (sans solliciter l'application Python), gere la compression, le load balancing entre plusieurs workers/ instances, et protege l'application des connexions lentes/malveillantes directes.

MID Q117. Pourquoi utiliser des variables d'environnement pour la configuration ?

Reponse. Pour separer la configuration (secrets, URLs de base de donnees, cles API) du code source, permettant de deployer le *me*me artefact/image sur plusieurs environnements (dev, staging, prod) sans jamais commiter de secrets dans le depot git – principe des *Twelve-Factor Apps*.

MID Q118. Que verifie typiquement un pipeline CI/CD pour une API Python ?

Reponse. Lint/formatage (ruff, black), typage statique (mypy), execution de la suite de tests (pytest), build de l'image Docker, scan de securite des dependances, puis deploiement automatique (souvent apres validation manuelle en production). L'objectif est de detecter les regressions avant qu'elles n'atteignent la production.

16. Questions sur tes projets

Pourquoi cette section compte le plus

30 a 40 % d'un entretien backend porte sur l'explication detaillee de tes propres projets (CV, GitHub). Prepare des reponses concretes et chiffrees – « j'ai reduit le temps de reponse de 800ms a 120ms en ajoutant du cache Redis » vaut toujours mieux qu'une reponse generique.

MID **Q119.** Pourquoi Django plutot que FastAPI (ou l'inverse) pour ce projet ?

Reponse. Prepare une reponse basee sur les besoins reels du projet : Django s'impose quand on a besoin d'un ecosysteme complet rapidement (admin, ORM mature, auth, formulaires) pour une application classique avec beaucoup de CRUD et de vues. FastAPI s'impose pour des APIs pures orientees performance/async, des microservices, ou une forte contrainte de validation de schema stricte (Pydantic) – typiquement, une API consommee par un frontend decouple ou d'autres services.

MID **Q120.** Pourquoi FastAPI plutot que Flask ?

Reponse. FastAPI apporte nativement : validation de types via Pydantic, documentation OpenAPI automatique, support async natif (ASGI), et injection de dependances integree. Flask est plus minimaliste (WSGI, synchrone par default), demandant des extensions tierces pour obtenir des fonctionnalites equivalentes.

MID **Q121.** Comment as-tu structure ton projet (architecture des dossiers/couches) ?

Reponse. Prepare une reponse concrete : separation routes/schemas (Pydantic)/models (ORM)/services (logique metier)/repositories (acces donnees), configuration centralisee, tests miroir de la structure source. Sois pret a justifier *pourquoi* cette structure (testabilite, lisibilite, isolation des couches) plutot que de la lister sans argumentation.

MID **Q122.** Comment geres-tu les exceptions dans ton API ?

Reponse. Prepare une reponse concrete : exceptions metier personnalisees, handler global (`@app.exception_handler` en FastAPI, middleware en Django) qui traduit chaque exception en reponse HTTP coherente (code de statut + message structure), sans jamais exposer de stack trace ou de details internes au client en production.

MID **Q123.** Comment valides-tu les donnees entrantes ?

Reponse. Prepare une reponse concrete : Pydantic (FastAPI) ou Serializers/Forms (Django/DRF) pour la validation de structure/type au plus tot (a la frontiere de l'API), plus des regles metier explicites dans la couche service pour les validations qui dependent de l'etat de la base (ex. unicite, coherence entre entites).

MID **Q124.** Comment securises-tu ton API ?

Reponse. Prepare une reponse concrete couvrant : authentication (JWT/OAuth2), autorisation par role/scope, validation stricte des entrees, HTTPS partout, CORS configure explicitement (pas de wildcard en prod), rate limiting, gestion des secrets via variables d'environnement (jamais en dur dans le code).

MID **Q125.** Comment testes-tu ton code ?

Reponse. Prepare une reponse concrete : pyramide de tests (unitaires sur la logique metier, tests d'integration sur les endpoints via TestClient/APIClient, quelques tests end-to-end critiques), utilisation de fixtures/mocks pour isoler les dependances externes, mesure de couverture pour reperer les zones non testees sans en faire un objectif chiffre absolu.

MID Q126. Comment deploies-tu ton application ?

Reponse. Prepare une reponse concrete : conteneurisation Docker, pipeline CI/CD (tests puis build puis deploiement), configuration via variables d'environnement, reverse proxy (Nginx) devant Gunicorn/Uvicorn, strategie de rollback en cas de probleme.

MID Q127. Pourquoi PostgreSQL plutot qu'un autre SGBD ?

Reponse. Prepare une reponse concrete : conformite ACID stricte, richesse des types (JSONB pour des donnees semi-structurees, types tableaux), extensions puissantes (pg_vector pour l'IA/RAG, pg_cron), excellent support des transactions concurrentes et de l'indexation avancee, tres bon eco-systeme avec les ORMs Python.

MID Q128. Pourquoi Redis dans ton architecture ?

Reponse. Prepare une reponse concrete : cas d'usage precis – cache de requetes couteuses, stockage de session, rate limiting, file de messages/taches (avec Celery ou en broker pour Kafka-like patterns), pub/sub pour de la notification temps reel. Justifie le choix par la latence (memoire, pas disque) plutot que « parce que c'est standard ».

SENIOR Q129. Comment evites-tu les requetes N+1 dans ton projet concret ?

Reponse. Prepare une reponse concrete avec un exemple reel : identification via profiling/logs SQL, correction avec `select_related/ prefetch_related` (Django) ou chargement explicite en batch (SQLAlchemy `joinedload/selectinload`), et si possible un chiffre avant/apres (nombre de requetes, temps de reponse).

SENIOR Q130. Comment ameliore-tu les performances globales de ton API ?

Reponse. Prepare une reponse concrete et hierarchisee : d'abord mesurer (profiling, APM) avant d'optimiser, puis leviers courants – indexation SQL adaptee, cache (Redis) sur les lectures couteuses et peu volatiles, pagination systematique, requetes async pour les I/O concurrentes, mise en cache HTTP (ETag, Cache-Control), et en dernier recours scaling horizontal.